Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 9

Disciplina: Tehnici de programare

Tema: ” Colectii: liste, tupluri, multimi, dictionare”

A efectuat : st. gr.SI – 262 Bodaci Bogdan

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

A1. Scheletul si prima lista.

Creează arsenal/privesc/enum.py. Întâi citești fișierele SUID într-o listă și o numeri. Un fișier de citire comun îți scoate liniile curate (fără goluri, fără comentarii):

```
import argparse, json, os

def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        ...

cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Pe ecran trebuie să vezi:

```
linii SUID: 14
```

De observat. Sunt 14 rânduri, dar unele se repetă. Lista păstrează tot, în ordine, inclusiv dublurile. Le strângem la pasul următor.

Am folosit pentru aceasta un return cu tip list pentru ca sa ne returneze toate randurile din filul suid.txt care contine toate aceste randuri:

```
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/gpasswd
/bin/mount
/bin/umount
/bin/su
/usr/bin/find
/usr/bin/vim.basic
/usr/bin/nmap
/bin/cp
/usr/bin/passwd
```

Si cu aceasta am facut acest cod prin perechea de azi, la inceput am facut prin for si if, care dupa am trecut la tip lista cu retrun pentru ca sa fie mai scurt intr-o linie:

Exemplul care lam facut inainte de aula:

```python
import argparse, json, os


def citeste_linii(cale):  1 usage
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        linii = []

        for linie in f:
            linie = linie.strip()

            if linie and not linie.startswith("#"):
                linii.append(linie)

        return linii


cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Exemplul care l-am facut dupa aula si exemplul usor:

```python
import argparse, json, os


def citeste_linii(cale):  1 usage
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        return [line for line in f if not line.startswith("#")]


cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Acesta este outputul la ambele:

```
C:\Users\Akicide\Desktop\Pycharm\.venv\Scripts\python.exe C:\Users\Akicide\Desktop\Pycharm\lab9\arsenal\privesc\enum.py
linii SUID: 14

Process finished with exit code 0
```

Care s-a facut de ogramada ori mai scurt, in loc de 6 linii, l-am facut intr-o linie.

La fel am observat ca daca schimb de la tipul list la tipul set outputul va fi diferit.

Iata tipul set:

```
import argparse, json, os


def citeste_linii(cale):  1 usage
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce lista liniilor .strip(), fara cele goale si fara cele cu #
        return {line for line in f if not line.startswith("#")}


cai = citeste_linii("start/suid.txt")
print("linii SUID:", len(cai))
```

Iata outputul:

```
C:\Users\Akicide\Desktop\Pycharm\.venv\Scripts\python.exe C:\Users\Akicide\Desktop\Pycharm\lab9\arsenal\privesc\test.py
linii SUID: 13

Process finished with exit code 0
```

In suid.txt este de 2 ori /usr/bin/passwd si in loc sa il citeasca de 2 ori pur si simplu o sa il ia odata si in loc de 14 o sa ne dea 13.

A2. Nume unice: de la lissta la multime

Numele fișierului e ce decide dacă e periculos (find, vim, nmap...). Scoate numele din fiecare cale și ține-le o singură dată.

```
def nume(cale):
    # TODO: intoarce ce e dupa ultimul "/"  (vezi rsplit)
    ...

nume_unice = {nume(c) for c in cai}    # set comprehension
print("SUID distincte:", len(nume_unice))
```

Pe ecran trebuie să vezi:

```
SUID distincte: 13
```

De observat. 14 rânduri, 13 nume distincte: mulțimea a aruncat dublura. Dacă îți dă tot 14, verifică funcția nume: probabil întoarce toată calea, nu doar numele.

Deci am folosit rsplit(), care incepe sa citeasca din dreapta in stanga, care in paranteza am dat “/” si 1 pentru ca sa dead oar un split de la dreapta la stanga, deci desparte lista care avea de exemplu usr/list/pswrd si il desparte in “usr/list”,”pswrd”, iar [-1] ia valoarea de la sfarsit care este “pswrd”, la care dupa trece prin nume_unice care face definite nume(c) si pentru fiecare c adica fiecare 14 caractere .

Aici este codul:

```
def nume(cale :{rsplit} ):  1 usage
    # TODO: intoarce ce e dupa ultimul "/"  (vezi rsplit)
    return cale.rsplit("/", 1)[-1]

nume_unice = {nume(c) for c in cai}  # set comprehension
print("SUID distincte:", len(nume_unice))
```

Aici este outputul:

```
C:\Users\Akicide\Desktop\Pycharm\.venv\Scripts\python.exe C:\Users\Akicide\Desktop\Pycharm\lab9\arsenal\privesc\enum.py
linii SUID: 14
SUID distincte: 13

Process finished with exit code 0
```

A3.Constatările SUID: listă de dicționare

Fiecare SUID devine o fișă: calea, numele, riscul. Periculos = numele e într-o mulțime de binare cunoscute ca scăpări (stil GTFOBins). Întorci o listă de dicționare, sortată, fără dubluri.

```
GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}

def suid_constatari(cale):
    cai = citeste_linii(cale)
    # TODO: pentru fiecare cale UNICA, in ordine sortata, un dict:
    #   {"cale": c, "nume": nume(c),
    #    "risc": "mare" daca nume(c) in GTFO, altfel "normal"}
    ...

suid = suid_constatari("start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]
print(len(suid), "SUID,", len(mari), "cu risc mare")
print(sorted(c["nume"] for c in mari))
```

Pe ecran trebuie să vezi:

```
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
```

Deci aici am facut asa ceva :

```python
def suid_constatari(cale):  1 usage
    cai = citeste_linii(cale)
    unice = sorted(set [str] (cai))
    # TODO: pentru fiecare cale UNICA, in ordine sortata, un dict:
    #   {"cale": c, "nume": nume(c),
    #    "risc": "mare" daca nume(c) in GTFO, altfel "normal"}
    return [
        {
        "cale": c,
        "nume": nume(c),
        "risc": "mare" if nume(c) in GTFO else "normal"
        }
        for c in unice
    ]


suid = suid_constatari("start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]
print(len(suid), "SUID,", len(mari), "cu risc mare")
print(sorted(c["nume"] for c in mari))
```

Am facut o variabila noua unice, care va sorta in seturi locatia cai, care este “start/suid.txt”, in loc sa fac un for si sa fac prima linie din TODO.

am facut un return pentru a fii returnate “cale”:c, “nume”: nume(c) si “risc”: returneaza “mare” daca numele este in GTFO, daca nu va fi “normal”.

Dupa for c in unice. Pentru a le lista mai departe.

Care dupa imi da acest output:



A4.Sudo si cron:tipul si gruparea

O linie din sudo.txt are forma „user : drept". O despachetezi într-un tuplu. O linie din crontab.txt are șase câmpuri de program, apoi userul și comanda. Fiecare sursă dă o listă de dicționare.

```
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)   # despachetare in tuplu
        # TODO: adauga {"user": user.strip(), "drept": drept.strip(),
        #   "risc": "mare" daca "NOPASSWD" in drept, altfel "normal"}
        ...
    return out

def cron_constatari(cale):
    # TODO: campuri = ln.split(); user = campuri[5];
    #   comanda = " ".join(campuri[6:]); dict cu user si comanda
    ...
```

Acum le grupezi pe toate într-un singur raport: un dicționar cu o cheie pe categorie, valoarea = lista ei.

```
def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

Deci am facut asa ceva:

```
def sudo_constatari(cale):  1 usage
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)   # despachetare in tuplu
        # TODO: adauga {"user": user.strip(), "drept": drept.strip(),
        #   "risc": "mare" daca "NOPASSWD" in drept, altfel "normal"}
        return [
            {
                "user": user.strip(),
                "drept": drept.strip(),
                "risc": "mare" if "NOPASSWD" in drept else "normal"
            }
        ]
    return out

def cron_constatari(cale):  1 usage
    # TODO: campuri = ln.split(); user = campuri[5];
    #   comanda = " ".join(campuri[6:]); dict cu user si comanda
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split()
        user = campuri[5]
        comanda = " ".join(campuri[6:])
        out.append({"user": user, "comanda": comanda})


def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

Deci in acesta am facut un return in care am facut niste obiecte “user”,”drept”,”risc”.

Pentru “user” imi va fi intors ca user fara spatiu.

Pentru “drept” imi va intors ca drept fara loc intre ele.

“risc” va fi intros ca “mare” daca “NOPASSWD” este in drept daca nu “normal”

Mai departe pentru cron_constatari(cale): amnotat campurile ca ln fara spatii.

User va fi a cincelea caracter din campuri.

Comanda va fi spatiu gol care a fost joined cu campuri de la valorea a 6 din table pana la utlime.

Out care a fost gold iam dat append pentru ca sa ii revina fiecare user la user si fiecare comanda la comanda si s as e adauge la fiacre table.

A5.Raportul: JSON si rezumat pe categorii

main() cu argparse: primește folderul capturilor și un comutator --json. Cu --json scoate JSON curat; fără el, un rezumat pe categorii (cu .items()) și salvează raportul pe disc.

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False)); return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din cron
    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")

if __name__ == "__main__":
    main()
```

Rulezi din rădăcina depozitului, cu -m, dând calea capturilor:

```
python -m arsenal.privesc.enum arsenal/privesc/start
```

Pe ecran trebuie să vezi:

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

DE REȚINUT. Patru colecții, fiecare la locul ei: lista ține fișierele în ordine, tuplul despachetează o linie, mulțimea scoate userii unici, dicționarul grupează raportul. Codul a ieșit scurt pentru că structura a fost aleasă bine.

Programul care merge:

```python
def main():  1 usage
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument( *name_or_flags: "--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False));
        return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    # TODO: utilizatorii implicati = multimea userilor din sudo REUNITA cu cei din cron
    user_sudo = {c["user"] for c in raport["sudo"]}
    user_cron = {c["user"] for c in raport["cron"]}
    utilizatori = user_sudo | user_cron
    print("utilizatori implicati:", ", ".join(sorted(utilizatori)))

    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")

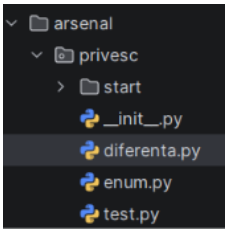
    if __name__ == "__main__":
        main()
```

Output:

```
(.venv) PS C:\Users\Akicide\Desktop\Pycharm\
enal/privesc/start
linii SUID: 14
SUID distincte: 13
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

Lucrarea B:

Am create un file .py nou si l-am numit diferenta.py in directory (privesc):



B1.Doua multimi: curat si acum.

```
def cai(fisier):
    # TODO: intoarce MULTIMEA cailor din fisier (foloseste citeste_linii din enum)
    ...

baza = cai("start/baza_suid.txt")
acum = cai("start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))
```

Pe ecran trebuie să vezi:

```
baza: 9  acum: 13
```

Aici e programul care merge:

```
from arsenal.privesc.enum import citeste_linii



def cai(fisier):  2 usages
    # TODO: intoarce MULTIMEA cailor din fisier (foloseste citeste_linii din enum)
    return set(citeste_linii(fisier))
```

Aici e outputul:

```
baza: 9  acum: 13
```

B2.Diferenta ce a aparut.

Ce e în „acum" dar nu în „bază" sunt fișierele SUID noi. Un singur operator, nu o buclă dublă.

```
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Pe ecran trebuie să vezi:

```
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

Aici e codul care lucreaza:

```
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Aici e outputul:

```
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

B3.Dictionar de risc si filtru final

Faci o fișă nume -> risc pentru cele noi (dict comprehension), apoi filtrezi doar periculoasele (list comprehension cu if). Refolosești nume() și GTFO din enum printr-un import.

```
from arsenal.privesc.enum import nume, GTFO

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Pe ecran trebuie să vezi:

```
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```

De observat. Toate cele patru apărute sunt periculoase: cineva a pus bit SUID pe find, nmap, cp și vim. Oricare dă root. Asta e exact genul de constatare pe care o scoți în față, nu o îngropi în raport.

Aici e codul care lucreaza:

```
from arsenal.privesc.enum import nume, GTFO

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Aici e outputul:

```
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```

B4.Un test care prinde regresia.

Scrie arsenal/privesc/test_diferenta.py cu un test mic, cu date scrise de mână, care verifică faptul cheie: ce e în acum și nu în bază iese corect. Rulează cu pytest și treacă fără rețea.

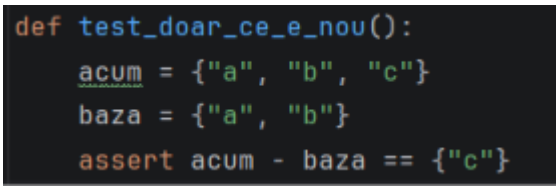
```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}

pytest -q arsenal/privesc/test_diferenta.py
```

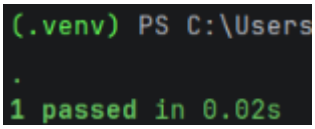
Pe ecran trebuie să vezi:

```
1 passed
```

Aici e codul care merge:



Aici e outputul:



CONCLUZII

În acest laborator am aplicat colecțiile Python (liste, seturi, dicționare) pe un caz practic real: enumerarea și analiza unor date de tip privilege escalation (binare SUID, drepturi sudo, intrări crontab).

Am văzut cum fiecare tip de colecție își are rolul potrivit:

- **liste** (`list`) — pentru a păstra liniile unui fișier, în ordinea citirii, inclusiv eventualele duplicate;
- **seturi** (`set`) — pentru a elimina rapid duplicatele (`set(cai)`) și pentru a face operații de tip diferență/reuniune între două colecții (`acum - baza`, `useri_sudo | useri_cron`) — util exact pentru a identifica ce e *nou* sau *comun* între două stări ale sistemului;
- **dicționare** (`dict`) — pentru a structura fiecare constatare (cale, nume, nivel de risc) într-o formă ușor de citit și de exportat ca JSON.

Pe partea practică, laboratorul a scos la iveală și câteva lecții utile dincolo de sintaxă:

- un `.strip()` uitat sau un `return` pus greșit în interiorul unei bucle pot produce rezultate aparent corecte (nicio eroare), dar silențios greșite — de aceea verificarea rezultatelor cu `print`/`repr()` la fiecare pas e esențială;
- rularea scripturilor ca modul (`python -m pachet.modul`) dintr-un director rădăcină, în loc de rulare directă dintr-un subfolder, evită conflicte de nume cu module din biblioteca standard (cazul `enum.py` vs. modulul `enum` din Python);
- separarea clară a logicii (citire → structurare → comparare → raportare) face codul mai ușor de testat bucată cu bucată, cum s-a văzut la depanarea pas-cu-pas a funcțiilor `suid_constatari`, `sudo_constatari` și `cron_constatari`.